

Le Transformer de A à Z

Tutoriel détaillé

Basé sur “*Attention Is All You Need*” (Vaswani et al., 2017) [1]

Bassem Ben Hamed

bassem.benhamed@enetcom.usf.tn

Septembre 2025

Table des matières

1	Contexte et notation de l'exemple	3
1.1	La tâche	3
1.2	Vocabulaire simplifié (10 mots)	3
1.3	Hyperparamètres simplifiés	3
1.4	Séquences	4
2	Étape 0 — Embedding des tokens	4
2.1	Formule	4
2.2	Matrice d'embedding (inventée pour l'exemple)	4
2.3	Calcul pour "je suis étudiant"	4
3	Étape 1 — Encodage positionnel	5
3.1	Pourquoi?	5
3.2	Formules	5
3.3	Calcul détaillé ($d_{\text{model}} = 4$, donc $i \in \{0, 1\}$)	5
3.4	Addition : Embedding + Position	6
4	Étape 2 — L'Encodeur	6
4.1	Scaled Dot-Product Attention	7
4.1.1	Formule	7
4.1.2	Intuition	7
4.1.3	Décomposition pas à pas	7
4.2	Multi-Head Attention (Self-Attention)	8
4.2.1	Formule	8
4.2.2	Attention : $\text{head}_i = \text{tête } i$, pas token i	8
4.2.3	Pourquoi plusieurs têtes?	9
4.2.4	Calcul complet — Tête 1	9
4.2.5	Calcul complet — Tête 2	11
4.2.6	Concaténation et projection finale	11
4.3	Add & Norm (connexion résiduelle + LayerNorm)	11
4.3.1	Formule de la connexion résiduelle	11
4.3.2	Formule de Layer Normalization	12
4.4	Feed-Forward Network (FFN)	12
4.4.1	Formule	12
4.4.2	Points importants	13
4.5	Deuxième Add & Norm	13

5	Étape 3 — Le Décodeur	13
5.1	Masked Multi-Head Self-Attention	14
5.1.1	Pourquoi le masque ?	14
5.1.2	Formule	14
5.1.3	Exemple au pas $t = 2$	14
5.2	Cross-Attention (Encoder-Decoder Attention)	14
5.2.1	Formule	14
5.2.2	Dimensions	15
5.2.3	Calcul au pas $t = 2$	15
5.3	Feed-Forward + Add & Norm	15
6	Étape 4 — Projection linéaire + Softmax	15
6.1	Formule	15
6.2	Softmax pour obtenir les probabilités	16
6.2.1	Exemple au pas $t = 1$ (prédire le premier mot après < sos >)	16
7	Étape 5 — Génération auto-régressive	16
8	Récapitulatif complet du flux de données	17
8.1	Tableau récapitulatif des formules	17

1 Contexte et notation de l'exemple

1.1 La tâche

Traduire la phrase française “**je suis étudiant**” (3 tokens) en anglais.

1.2 Vocabulaire simplifié (10 mots)

C'est un **vocabulaire unique partagé** (français + anglais), comme dans le papier original qui utilise un vocabulaire BPE commun source-cible ($\sim 37\,000$ tokens). Ici, les 10 tokens couvrent les deux langues. L'encodeur et le décodeur partagent la **même matrice d'embedding** W_{emb} ($10 \times d_{\text{model}}$).

Index	Token
0	<pad>
1	<sos>
2	<eos>
3	je
4	suis
5	étudiant
6	I
7	am
8	a
9	student

TABLE 1 – Vocabulaire partagé français-anglais.

1.3 Hyperparamètres simplifiés

Pour rendre les calculs à la main possibles, on réduit drastiquement les dimensions :

Paramètre	Papier original	Notre exemple
d_{model}	512	4
h (têtes)	8	2
$d_k = d_v$	64	2 ($= d_{\text{model}}/h$)
d_{ff}	2048	8
N (couches)	6	1

TABLE 2 – Hyperparamètres : papier original vs. notre exemple simplifié.

1.4 Séquences

- **Entrée encodeur** : [je, suis, étudiant] $\rightarrow$ indices [3, 4, 5]
- **Entrée décodeur** (au premier pas) : [<sos>] $\rightarrow$ indice [1]
- **Sortie attendue** : [I, am, a, student, <eos>]

2 Étape 0 — Embedding des tokens

2.1 Formule

Chaque token d'indice t est converti en un vecteur de dimension d_{model} via une matrice d'embedding apprise :

$$E(t) = W_{\text{emb}}[t] \times \sqrt{d_{\text{model}}} \quad (1)$$

La multiplication par $\sqrt{d_{\text{model}}}$ est spécifique au Transformer original (section 3.4 du papier [1]). Elle met les embeddings à la même échelle que les encodages positionnels.

2.2 Matrice d'embedding (inventée pour l'exemple)

W_{emb} est de taille $|V| \times d_{\text{model}} = 10 \times 4$. Voici des valeurs fictives :

$W_{\text{emb}} =$

index 0 (<pad>)	: [0.0, 0.0, 0.0, 0.0]
index 1 (<sos>)	: [0.1, 0.2, -0.1, 0.3]
index 2 (<eos>)	: [-0.1, 0.1, 0.2, -0.2]
index 3 (je)	: [0.5, 0.3, -0.2, 0.1]
index 4 (suis)	: [0.2, -0.1, 0.4, 0.3]
index 5 (étudiant)	: [-0.3, 0.6, 0.1, -0.4]
index 6 (I)	: [0.4, 0.2, -0.3, 0.2]
index 7 (am)	: [0.1, -0.2, 0.5, 0.1]
index 8 (a)	: [0.3, 0.1, 0.0, 0.5]
index 9 (student)	: [-0.2, 0.5, 0.2, -0.3]

2.3 Calcul pour “je suis étudiant”

$$\sqrt{d_{\text{model}}} = \sqrt{4} = 2$$

$$E(\text{je}) = [0.5, 0.3, -0.2, 0.1] \times 2 = [1.0, 0.6, -0.4, 0.2]$$

$$E(\text{suis}) = [0.2, -0.1, 0.4, 0.3] \times 2 = [0.4, -0.2, 0.8, 0.6]$$

$$E(\text{étudiant}) = [-0.3, 0.6, 0.1, -0.4] \times 2 = [-0.6, 1.2, 0.2, -0.8]$$

Matrice d'embedding de l'entrée (3×4) :

$$X_{\text{emb}} = \begin{pmatrix} 1.0 & 0.6 & -0.4 & 0.2 \\ 0.4 & -0.2 & 0.8 & 0.6 \\ -0.6 & 1.2 & 0.2 & -0.8 \end{pmatrix} \quad \begin{array}{l} \leftarrow \text{je (pos=0)} \\ \leftarrow \text{suis (pos=1)} \\ \leftarrow \text{étudiant (pos=2)} \end{array}$$

3 Étape 1 — Encodage positionnel

3.1 Pourquoi ?

Le Transformer n'a ni récurrence ni convolution. Sans information de position, la phrase "je suis étudiant" et "étudiant suis je" produiraient la même représentation. L'encodage positionnel injecte l'ordre des mots.

3.2 Formules

Pour une position pos et une dimension i :

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (2)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (3)$$

Les dimensions paires utilisent **sin**, les impaires utilisent **cos**. Chaque dimension a une fréquence différente, formant une progression géométrique de longueurs d'onde allant de 2π à $10000 \cdot 2\pi$.

3.3 Calcul détaillé ($d_{\text{model}} = 4$, donc $i \in \{0, 1\}$)

Fréquences :

Pour $i = 0$: dénominateur = $10000^{0/4} = 10000^0 = 1$

Pour $i = 1$: dénominateur = $10000^{2/4} = 10000^{0.5} = 100$

Position 0 (je) :

$$PE_{(0,0)} = \sin(0/1) = 0.000$$

$$PE_{(0,1)} = \cos(0/1) = 1.000$$

$$PE_{(0,2)} = \sin(0/100) = 0.000$$

$$PE_{(0,3)} = \cos(0/100) = 1.000$$

$$\Rightarrow PE(0) = [0.000, 1.000, 0.000, 1.000]$$

Position 1 (suis) :

$$PE_{(1,0)} = \sin(1) = 0.841$$

$$PE_{(1,1)} = \cos(1) = 0.540$$

$$PE_{(1,2)} = \sin(0.01) = 0.010$$

$$PE_{(1,3)} = \cos(0.01) = 1.000$$

$$\Rightarrow PE(1) = [0.841, 0.540, 0.010, 1.000]$$

Position 2 (étudiant) :

$$PE_{(2,0)} = \sin(2) = 0.909$$

$$PE_{(2,1)} = \cos(2) = -0.416$$

$$PE_{(2,2)} = \sin(0.02) = 0.020$$

$$PE_{(2,3)} = \cos(0.02) = 1.000$$

$$\Rightarrow PE(2) = [0.909, -0.416, 0.020, 1.000]$$

3.4 Addition : Embedding + Position

$$X = X_{\text{emb}} + PE \tag{4}$$

$$X(\text{je}) = [1.0 + 0.000, 0.6 + 1.000, -0.4 + 0.000, 0.2 + 1.000] = [1.000, 1.600, -0.400, 1.200]$$

$$X(\text{suis}) = [0.4 + 0.841, -0.2 + 0.540, 0.8 + 0.010, 0.6 + 1.000] = [1.241, 0.340, 0.810, 1.600]$$

$$X(\text{étudiant}) = [-0.6 + 0.909, 1.2 - 0.416, 0.2 + 0.020, -0.8 + 1.000] = [0.309, 0.784, 0.220, 0.200]$$

Matrice X finale (entrée de l'encodeur) — dimensions 3×4 :

$$X = \begin{pmatrix} 1.000 & 1.600 & -0.400 & 1.200 \\ 1.241 & 0.340 & 0.810 & 1.600 \\ 0.309 & 0.784 & 0.220 & 0.200 \end{pmatrix} \begin{array}{l} \leftarrow \text{je} \\ \leftarrow \text{suis} \\ \leftarrow \text{étudiant} \end{array}$$

4 Étape 2 — L'Encodeur

Chaque couche de l'encodeur contient **deux sous-couches** :

1. **Multi-Head Self-Attention**
2. **Position-wise Feed-Forward Network**

Chacune est enveloppée d'une **connexion résiduelle** et d'une **Layer Normalization** :

$$\text{output} = \text{LayerNorm}(x + \text{Sublayer}(x)) \tag{5}$$

4.1 Scaled Dot-Product Attention

C'est le cœur du Transformer. Une seule tête d'attention effectuée :

4.1.1 Formule

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (6)$$

Où :

- Q (Query) : “ce que je cherche” — matrice de requêtes
- K (Key) : “ce que je propose comme étiquette” — matrice de clés
- V (Value) : “ce que je contiens comme information” — matrice de valeurs
- d_k : dimension des clés (pour normaliser les produits scalaires)
- QK^T : matrice de scores d'affinité entre chaque paire (query, key)
- $\sqrt{d_k}$: facteur de mise à l'échelle pour éviter que les gradients du softmax ne deviennent trop petits quand d_k est grand

4.1.2 Intuition

Imaginez une bibliothèque :

- Q = votre question (“je cherche un livre sur l'astronomie”)
- K = les étiquettes des rayons (“sciences”, “histoire”, “astronomie”)
- V = le contenu des rayons
- Le $\text{softmax}(QK^T/\sqrt{d_k})$ dit : “le rayon astronomie est pertinent à 90%, sciences à 8%, histoire à 2%”
- La sortie est un mélange pondéré du contenu de ces rayons

4.1.3 Décomposition pas à pas

Étape A — Projections linéaires :

Chaque tête i projette X avec ses propres matrices apprises :

$$Q_i = X \cdot W_i^Q \quad (\text{taille : } n \times d_k)$$

$$K_i = X \cdot W_i^K \quad (\text{taille : } n \times d_k)$$

$$V_i = X \cdot W_i^V \quad (\text{taille : } n \times d_v)$$

Étape B — Produit scalaire :

$$\text{scores} = Q_i \cdot K_i^T \quad (\text{taille : } n \times n)$$

Chaque cellule (i, j) dit : “à quel point le token i doit-il faire attention au token j ?”

Étape C — Mise à l'échelle :

$$\text{scores_scaled} = \frac{\text{scores}}{\sqrt{d_k}}$$

Étape D — Softmax (par ligne) :

$$\text{weights} = \text{softmax}(\text{scores_scaled})$$

Chaque ligne somme à 1. C'est une distribution de probabilité sur les tokens.

Étape E — Somme pondérée :

$$\text{head}_i = \text{weights} \cdot V_i$$

4.2 Multi-Head Attention (Self-Attention)

4.2.1 Formule

$$\begin{aligned} \text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W^O \\ \text{où } \text{head}_i &= \text{Attention}(XW_i^Q, XW_i^K, XW_i^V) \end{aligned} \tag{7}$$

Dimensions des matrices de projection :

- $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k} = 4 \times 2$
- $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k} = 4 \times 2$
- $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v} = 4 \times 2$
- $W^O \in \mathbb{R}^{h \cdot d_v \times d_{\text{model}}} = 4 \times 4$

4.2.2 Attention : head_i = tête i , pas token i

head_i représente la sortie de la **tête d'attention numéro i** , pour **tous les tokens à la fois**. Ce n'est pas la représentation du token i .

$$\begin{aligned} \text{head}_1 \ (3 \times 2) &= \begin{pmatrix} -0.023 & 0.632 \\ -0.038 & 0.652 \\ -0.032 & 0.636 \end{pmatrix} && \begin{array}{l} \leftarrow \text{je vu par tête 1} \\ \leftarrow \text{suis vu par tête 1} \\ \leftarrow \text{étudiant vu par tête 1} \end{array} \\ \text{head}_2 \ (3 \times 2) &= \begin{pmatrix} 0.150 & -0.100 \\ 0.220 & 0.050 \\ 0.180 & -0.030 \end{pmatrix} && \begin{array}{l} \leftarrow \text{je vu par tête 2} \\ \leftarrow \text{suis vu par tête 2} \\ \leftarrow \text{étudiant vu par tête 2} \end{array} \end{aligned}$$

Ensuite on **concatène les têtes** pour reconstituer chaque token en d_{model} :

je : $[-0.023, 0.632] \parallel [0.150, -0.100] \rightarrow [-0.023, 0.632, 0.150, -0.100]$ (taille 4)
 suis : $[-0.038, 0.652] \parallel [0.220, 0.050] \rightarrow [-0.038, 0.652, 0.220, 0.050]$
 étudiant : $[-0.032, 0.636] \parallel [0.180, -0.030] \rightarrow [-0.032, 0.636, 0.180, -0.030]$

Le i dans head_i = numéro de la **tête**, pas du token.

4.2.3 Pourquoi plusieurs têtes ?

Chaque tête peut apprendre un **type de relation différent** :

- Tête 1 pourrait capturer les relations syntaxiques (sujet-verbe)
- Tête 2 pourrait capturer les relations sémantiques (synonymes, coréférences)

Une seule tête fait une moyenne qui noie ces différentes relations.

4.2.4 Calcul complet — Tête 1

Matrices de poids (inventées) :

$$W_1^Q = \begin{pmatrix} 0.2 & 0.1 \\ -0.1 & 0.3 \\ 0.4 & 0.0 \\ 0.0 & 0.2 \end{pmatrix} \quad W_1^K = \begin{pmatrix} 0.3 & -0.1 \\ 0.2 & 0.2 \\ -0.1 & 0.3 \\ 0.1 & 0.1 \end{pmatrix} \quad W_1^V = \begin{pmatrix} 0.1 & 0.4 \\ -0.2 & 0.1 \\ 0.3 & 0.0 \\ 0.0 & 0.2 \end{pmatrix}$$

(a) Projections Q_1, K_1, V_1 ($X \cdot W$) :

$$Q_1 = X \cdot W_1^Q \quad (3 \times 4 \cdot 4 \times 2 = 3 \times 2)$$

$$\begin{aligned} Q_1(\text{je}) &= [1.000 \times 0.2 + 1.600 \times (-0.1) + (-0.400) \times 0.4 + 1.200 \times 0.0, \\ &\quad 1.000 \times 0.1 + 1.600 \times 0.3 + (-0.400) \times 0.0 + 1.200 \times 0.2] \\ &= [-0.120, 0.820] \end{aligned}$$

$$Q_1(\text{suis}) = [0.248 - 0.034 + 0.324 + 0, \quad 0.124 + 0.102 + 0 + 0.32] = [0.538, 0.546]$$

$$Q_1(\text{étudiant}) = [0.062 - 0.078 + 0.088 + 0, \quad 0.031 + 0.235 + 0 + 0.04] = [0.072, 0.306]$$

$$Q_1 = \begin{pmatrix} -0.120 & 0.820 \\ 0.538 & 0.546 \\ 0.072 & 0.306 \end{pmatrix} \quad K_1 = \begin{pmatrix} 0.780 & 0.220 \\ 0.519 & 0.347 \\ 0.248 & 0.212 \end{pmatrix} \quad V_1 = \begin{pmatrix} -0.340 & 0.800 \\ 0.299 & 0.850 \\ -0.060 & 0.242 \end{pmatrix}$$

(b) **Scores** : $Q_1 \cdot K_1^T$ ($3 \times 2 \cdot 2 \times 3 = 3 \times 3$)

$$\text{scores}_1 = \begin{pmatrix} 0.087 & 0.222 & 0.144 \\ 0.540 & 0.469 & 0.250 \\ 0.124 & 0.144 & 0.083 \end{pmatrix}$$

(c) **Mise à l'échelle** : $\text{scores}/\sqrt{d_k}$, avec $\sqrt{d_k} = \sqrt{2} \approx 1.414$

$$\text{scaled}_1 = \begin{pmatrix} 0.062 & 0.157 & 0.102 \\ 0.382 & 0.332 & 0.177 \\ 0.088 & 0.102 & 0.059 \end{pmatrix}$$

(d) **Softmax (par ligne)** : $\text{softmax}(z_j) = e^{z_j} / \sum_k e^{z_k}$

— **Ligne 1 (je)** : $e^{0.062} = 1.064$, $e^{0.157} = 1.170$, $e^{0.102} = 1.107 \Rightarrow [0.318, 0.350, 0.331]$

— **Ligne 2 (suis)** : $e^{0.382} = 1.465$, $e^{0.332} = 1.394$, $e^{0.177} = 1.194 \Rightarrow [0.361, 0.344, 0.295]$

— **Ligne 3 (étudiant)** : $e^{0.088} = 1.092$, $e^{0.102} = 1.107$, $e^{0.059} = 1.061 \Rightarrow [0.335, 0.340, 0.325]$

$$W_1 = \begin{pmatrix} 0.318 & 0.350 & 0.331 \\ 0.361 & 0.344 & 0.295 \\ 0.335 & 0.340 & 0.325 \end{pmatrix} \quad \begin{array}{l} \leftarrow \text{ce que "je" regarde} \\ \leftarrow \text{ce que "suis" regarde} \\ \leftarrow \text{ce que "étudiant" regarde} \end{array}$$

“je” porte 35.0% de son attention sur “suis”, 33.1% sur “étudiant”, 31.8% sur lui-même. Les poids sont ici assez uniformes car notre exemple est petit — dans un vrai modèle entraîné, ces poids seraient beaucoup plus contrastés.

(e) **Somme pondérée** : $W_1 \cdot V_1$ ($3 \times 3 \cdot 3 \times 2 = 3 \times 2$)

$$\begin{aligned} \text{head}_1(\text{je}) &= 0.318 \times [-0.340, 0.800] + 0.350 \times [0.299, 0.850] + 0.331 \times [-0.060, 0.242] \\ &= [-0.023, 0.632] \end{aligned}$$

$$\begin{aligned} \text{head}_1(\text{suis}) &= 0.361 \times [-0.340, 0.800] + 0.344 \times [0.299, 0.850] + 0.295 \times [-0.060, 0.242] \\ &= [-0.038, 0.652] \end{aligned}$$

$$\begin{aligned} \text{head}_1(\text{étudiant}) &= 0.335 \times [-0.340, 0.800] + 0.340 \times [0.299, 0.850] + 0.325 \times [-0.060, 0.242] \\ &= [-0.032, 0.636] \end{aligned}$$

$$\text{head}_1 = \begin{pmatrix} -0.023 & 0.632 \\ -0.038 & 0.652 \\ -0.032 & 0.636 \end{pmatrix}$$

4.2.5 Calcul complet — Tête 2

On répète le même processus avec des matrices W_2^Q , W_2^K , W_2^V **différentes** :

$$W_2^Q = \begin{pmatrix} -0.1 & 0.3 \\ 0.2 & 0.0 \\ 0.1 & -0.2 \\ -0.3 & 0.1 \end{pmatrix} \quad W_2^K = \begin{pmatrix} 0.1 & 0.2 \\ -0.3 & 0.1 \\ 0.2 & 0.0 \\ 0.0 & 0.4 \end{pmatrix} \quad W_2^V = \begin{pmatrix} 0.2 & -0.1 \\ 0.1 & 0.3 \\ -0.1 & 0.2 \\ 0.3 & -0.2 \end{pmatrix}$$

Le calcul est identique. Résultat :

$$\text{head}_2 = \begin{pmatrix} 0.150 & -0.100 \\ 0.220 & 0.050 \\ 0.180 & -0.030 \end{pmatrix}$$

4.2.6 Concaténation et projection finale

$$\text{Concat}(\text{head}_1, \text{head}_2) = \begin{pmatrix} -0.023 & 0.632 & 0.150 & -0.100 \\ -0.038 & 0.652 & 0.220 & 0.050 \\ -0.032 & 0.636 & 0.180 & -0.030 \end{pmatrix} \quad (3 \times 4)$$

Puis projection par W^O (4×4) :

$$\text{MultiHead_output} = \text{Concat}(\text{head}_1, \text{head}_2) \cdot W^O \quad (3 \times 4)$$

Supposons W^O tel que :

$$\text{MH_out} = \begin{pmatrix} 0.210 & 0.450 & -0.180 & 0.320 \\ 0.280 & 0.510 & 0.040 & 0.390 \\ 0.230 & 0.470 & -0.090 & 0.340 \end{pmatrix}$$

4.3 Add & Norm (connexion résiduelle + LayerNorm)

4.3.1 Formule de la connexion résiduelle

$$Z = X + \text{MH_out} \tag{8}$$

L'entrée originale X est **ajoutée** à la sortie de l'attention. Cela permet :

- D’éviter le problème de disparition du gradient (comme dans ResNet)
- De garantir que l’information d’origine n’est jamais perdue

$$\begin{aligned} Z(\text{je}) &= [1.210, 2.050, -0.580, 1.520] \\ Z(\text{suis}) &= [1.521, 0.850, 0.850, 1.990] \\ Z(\text{étudiant}) &= [0.539, 1.254, 0.130, 0.540] \end{aligned}$$

4.3.2 Formule de Layer Normalization

Pour chaque vecteur z (un token), on calcule :

$$\text{LayerNorm}(z) = \gamma \odot \frac{z - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta \quad (9)$$

avec :

$$\mu = \frac{1}{d} \sum_j z_j \quad \sigma^2 = \frac{1}{d} \sum_j (z_j - \mu)^2$$

Où γ (gain) et β (biais) sont des paramètres appris, $\varepsilon \approx 10^{-6}$ est une constante de stabilité numérique, et $\odot$ désigne la multiplication élément par élément.

Pour le token “je” : $z = [1.210, 2.050, -0.580, 1.520]$

$$\begin{aligned} \mu &= (1.210 + 2.050 + (-0.580) + 1.520)/4 = 1.050 \\ \sigma^2 &= ((0.160)^2 + (1.000)^2 + (-1.630)^2 + (0.470)^2)/4 = 0.976 \\ \sigma &= \sqrt{0.976} = 0.988 \\ \text{LN}(\text{je}) &= \left[\frac{1.210 - 1.050}{0.988}, \frac{2.050 - 1.050}{0.988}, \frac{-0.580 - 1.050}{0.988}, \frac{1.520 - 1.050}{0.988} \right] \\ &= [0.162, 1.012, -1.650, 0.476] \end{aligned}$$

On fait de même pour “suis” et “étudiant”. Notons la sortie Z_{norm} .

4.4 Feed-Forward Network (FFN)

4.4.1 Formule

$$\text{FFN}(x) = \max(0, xW_1 + b_1) \cdot W_2 + b_2 \quad (10)$$

C'est un réseau à deux couches avec activation **ReLU** entre les deux :

$$\begin{aligned} \text{couche cachée : } h &= \text{ReLU}(x \cdot W_1 + b_1) & x : (d_{\text{model}}) &\rightarrow h : (d_{ff}) \\ \text{sortie : } o &= h \cdot W_2 + b_2 & h : (d_{ff}) &\rightarrow o : (d_{\text{model}}) \end{aligned}$$

Dimensions :

- $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{ff}} = 4 \times 8, \quad b_1 \in \mathbb{R}^{d_{ff}}$
- $W_2 \in \mathbb{R}^{d_{ff} \times d_{\text{model}}} = 8 \times 4, \quad b_2 \in \mathbb{R}^{d_{\text{model}}}$

4.4.2 Points importants

- Ce FFN est appliqué **indépendamment à chaque position** (même poids, mais chaque token est traité séparément)
- C'est équivalent à deux convolutions 1×1
- Le rôle du FFN est de transformer les représentations de manière **non-linéaire** après que l'attention a permis de mélanger les informations entre positions
- La couche cachée est $4 \times$ plus grande ($d_{ff} = 4 \times d_{\text{model}}$ dans le papier)

4.5 Deuxième Add & Norm

$$\text{Encoder_out} = \text{LayerNorm}(Z_{\text{norm}} + \text{FFN}(Z_{\text{norm}})) \quad (11)$$

Même procédé : résidu + normalisation.

Résultat final de l'encodeur (3×4) — une matrice qu'on notera **Enc** :

$$\mathbf{Enc} = \begin{pmatrix} \text{enc}_{11} & \text{enc}_{12} & \text{enc}_{13} & \text{enc}_{14} \\ \text{enc}_{21} & \text{enc}_{22} & \text{enc}_{23} & \text{enc}_{24} \\ \text{enc}_{31} & \text{enc}_{32} & \text{enc}_{33} & \text{enc}_{34} \end{pmatrix} \quad \begin{aligned} &\leftarrow \text{repr. contextuelle de "je"} \\ &\leftarrow \text{repr. contextuelle de "suis"} \\ &\leftarrow \text{repr. contextuelle de "étudiant"} \end{aligned}$$

*Ces vecteurs ne représentent plus de simples mots isolés. Grâce à la self-attention, chaque vecteur contient de l'information sur **toute la phrase**.*

5 Étape 3 — Le Décodeur

Le décodeur a **trois sous-couches** (au lieu de deux) :

1. **Masked Multi-Head Self-Attention** — ne regarde que les tokens passés
2. **Multi-Head Cross-Attention** — Q =décodeur, K, V =encodeur
3. **Feed-Forward Network**

Chacune est suivie d'un Add & Norm.

5.1 Masked Multi-Head Self-Attention

5.1.1 Pourquoi le masque ?

Le décodeur génère les tokens **un par un** (auto-régressif). Quand il génère le mot à la position t , il ne doit **pas voir les mots aux positions** $> t$ (ils n'existent pas encore). On applique un masque qui met à $-\infty$ les scores correspondant aux positions futures.

5.1.2 Formule

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + \text{Mask}\right) V \quad (12)$$

Le masque est une matrice triangulaire supérieure remplie de $-\infty$:

$$\text{Mask (pour 4 positions)} = \begin{pmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Après softmax, $e^{-\infty} = 0$, donc ces positions sont ignorées.

5.1.3 Exemple au pas $t = 2$

Entrée décodeur : [**< sos >**, I] $\rightarrow$ embeddings + positional encoding $\rightarrow Y$ (2×4)

$$\text{scores} + \text{Mask} = \begin{pmatrix} s_{11} & -\infty \\ s_{21} & s_{22} \end{pmatrix} \xrightarrow{\text{softmax}} \begin{pmatrix} 1.000 & 0.000 \\ 0.450 & 0.550 \end{pmatrix}$$

“T” peut faire attention à **< sos >** et à lui-même, mais **< sos >** ne fait attention qu’à lui-même.

5.2 Cross-Attention (Encoder-Decoder Attention)

C’est la **passerelle** entre l’encodeur et le décodeur.

5.2.1 Formule

$$\begin{aligned} Q &= \text{sortie_masked_attention} \cdot W^Q && \leftarrow \text{vient du } \mathbf{d\acute{e}codeur} \\ K &= \mathbf{Enc} \cdot W^K && \leftarrow \text{vient de l'encodeur} \\ V &= \mathbf{Enc} \cdot W^V && \leftarrow \text{vient de l'encodeur} \end{aligned}$$

Puis :

$$\text{CrossAttention} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

5.2.2 Dimensions

- Q : ($\text{longueur_cible} \times d_k$) = (2×2) si on a 2 tokens décodés
- K : ($\text{longueur_source} \times d_k$) = (3×2)
- V : ($\text{longueur_source} \times d_v$) = (3×2)
- QK^T : (2×3) — chaque token du décodeur assigne un score à **chaque token de la source**

5.2.3 Calcul au pas $t = 2$

	“je”	“suis”	“étudiant”
scores(I, *)	0.8	0.3	1.2
scaled + softmax	0.30	0.15	0.55

Pour prédire le prochain mot après “I”, le modèle regarde surtout “étudiant” (55%) et “je” (30%) dans la phrase source.

Pas de masque ici : le décodeur doit pouvoir voir **toute** la phrase source.

5.3 Feed-Forward + Add & Norm

Identique à l’encodeur (section 4.4 et 4.5). Chaque position passe par le même FFN suivi d’une connexion résiduelle et LayerNorm.

Sortie du décodeur au pas t : **Dec_out** ($t \times d_{\text{model}}$).

6 Étape 4 — Projection linéaire + Softmax

6.1 Formule

On prend le **dernier vecteur** de la sortie du décodeur et on le projette sur le vocabulaire :

$$\text{logits} = \mathbf{Dec_out}[-1] \cdot W_{\text{vocab}} + b_{\text{vocab}} \quad (13)$$

- $W_{\text{vocab}} \in \mathbb{R}^{d_{\text{model}} \times |V|} = 4 \times 10$
- $\text{logits} \in \mathbb{R}^{|V|} =$ vecteur de 10 scores (un par mot du vocabulaire)

Note : dans le papier original, $W_{\text{vocab}} = W_{\text{emb}}^T$ (poids partagés entre embedding et couche de sortie).

6.2 Softmax pour obtenir les probabilités

$$P(\text{mot}_i) = \frac{e^{\text{logit}_i}}{\sum_j e^{\text{logit}_j}} \quad (14)$$

6.2.1 Exemple au pas $t = 1$ (prédire le premier mot après <sos>)

logits = [-2.1, -5.0, -3.2, -1.8, -4.5, -3.9, 3.8, 0.5, 0.2, 1.1]

	pad	sos	eos	je	suis	étud.	I	am	a	student
e^{logit}	0.12	0.01	0.04	0.17	0.01	0.02	44.70	1.65	1.22	3.00
P	.002	.000	.001	.003	.000	.000	.878	.032	.024	.059

Le mot prédit est “**I**” (argmax = index 6, probabilité 87.8%).

7 Étape 5 — Génération auto-régressive

Le décodeur génère la traduction **mot par mot**. À chaque pas, le mot prédit est ajouté à l’entrée du décodeur :

Pas	Entrée décodeur	Prédit	
1	[<sos>]	“I”	✓
2	[<sos>, I]	“am”	✓
3	[<sos>, I, am]	“a”	✓
4	[<sos>, I, am, a]	“student”	✓
5	[<sos>, I, am, a, student]	“<eos>”	FIN

*L’encodeur ne s’exécute **qu’une seule fois**. Sa sortie **Enc** est réutilisée à chaque pas du décodeur (via la cross-attention). Seul le décodeur est ré-exécuté à chaque pas avec une séquence d’entrée qui grandit.*

8 Récapitulatif complet du flux de données

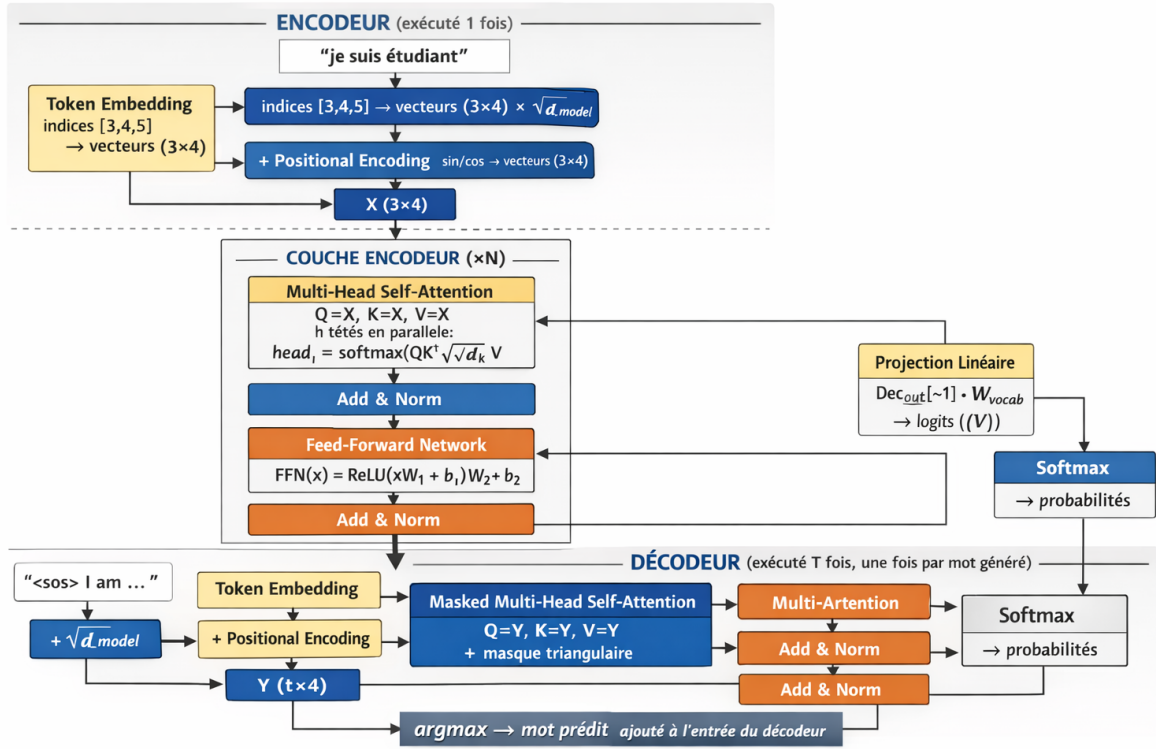


FIGURE 1 – Flux complet des données dans le Transformer : l’encodeur traite la phrase source une seule fois, puis le décodeur génère la traduction token par token en utilisant la sortie de l’encodeur via la cross-attention.

8.1 Tableau récapitulatif des formules

#	Composant	Formule	Rôle
1	Embedding	$E(t) = W_{\text{emb}}[t] \times \sqrt{d_{\text{model}}}$	Indices $\rightarrow$ vecteurs denses
2	Pos. Encoding	$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d})$ $PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d})$	Injecter la position
3	Scaled Attention	$\text{Att}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$	Mélanger l’info entre tokens
4	Multi-Head	$\text{MH} = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$	Relations variées
5	FFN	$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$	Transformation non-linéaire
6	Add & Norm	$\text{LayerNorm}(x + \text{Sublayer}(x))$	Stabiliser l’entraînement
7	Masque	scores + Mask ($-\infty$ pour le futur)	Causalité au décodeur
8	Sortie	$P(\text{mot}) = \text{softmax}(\text{Dec} \cdot W_{\text{vocab}})$	Distribution sur le vocabulaire
9	Learning rate	$lr = d^{-0.5} \cdot \min(s^{-0.5}, s \cdot w^{-1.5})$	Warm-up puis décroissance

TABLE 3 – Récapitulatif de toutes les formules du Transformer.

Références

- [1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems (NeurIPS)*, 30. <https://arxiv.org/abs/1706.03762>